192525074_RAVURU PRANATHI_CO1_AT1

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO1: Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships. (BL1, BL2)

Activity Tool : Short Answer Text(High) Assessment Tool

Weightage: 40%

QUESTIONS		
Q. No	**Question**	**Bloom 's Level**
1	Explain schema and instance with examples. Distinguish between logical schema, physical schema, and view schema.	BL1 – Remember
2	Identify the entities, attributes, and relationships for a Bank Management System and construct its ER diagram.	BL2 – Understand
3	Explain the Extended ER (EER) model. Describe the concepts of generalization, specialization, and aggregation.	BL2 – Understand
4	Design an EER diagram for a Hospital Management System incorporating inheritance and weak entities.	BL2 – Understand
5	Explain the role of a DBA (Database Administrator) and the responsibilities associated with the position.	BL1 – Remember

Code of Conduct*:*

I **R.Pranathi(192525076)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Pranathi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	30%	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Linkages	25%	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Organization	20%	Correctly applies concepts to solve the question/ problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Integration of Literature	15%	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity	10%	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1.Schema

Definition

A **schema** is the blueprint or logical design of a database. It specifies the structure of the database, including tables, attributes (columns), data types, relationships, and constraints. The schema is defined when the database is created and changes very rarely.

Example:

Consider a student database.

1.Student Table Schema

Atributes	Datatypes
Student_ID	Integer
Name	Varchar(50)
Department	Varchar(50)
Age	Integer

The schema can be represented as:

Student(Student_ID, Name, Department, Age)

This defines the structure of the Student table.

2. Instance

Definition

An **instance** is the actual data stored in the database at a particular point in time. Unlike the schema, the instance changes whenever records are inserted, updated, or deleted.

Example

Student_ID	Name	Department	Age
101	Ayesha	CSE	20
102	Rahul	ECE	21
103	Priya	IT	20

This table represents one instance of the Student database.

a. Logical Schema

Definition

The logical schema describes the logical structure of the database. It defines tables, attributes, relationships, primary keys, and foreign keys without considering how the data is physically stored.

Example

Student(Student_ID, Name, Department, Age)

Course(Course_ID, Course_Name, Credits)

The relationship between Student and Course is also defined in the logical schema.

Features

- Defines tables and attributes.
- Specifies relationships among tables.
- Independent of physical storage.
- Used by database designers.

b. Physical Schema

Definition

The physical schema describes how the database is physically stored on storage devices. It includes file organization, indexes, storage locations, and access methods.

Example

- Student records are stored in a file named **student.dat**.
- An index is created on **Student_ID** for faster searching.
- Data is stored in disk blocks.

Features

- Specifies storage details.
- Improves database performance.
- Managed by the DBMS.

c. View Schema (External Schema)

Definition

A view schema defines how different users view the database. It displays only the required information and hides unnecessary or confidential data.

Example:

Original Student Table

Student_ID	Name	Department	Age	Phone
101	Pranathi	CSE	20	9876543210

Faculty View

Student_ID	Name	Department
101	Pranathi	CSE

In this view, the **Age** and **Phone** fields are hidden.

Features

- Provides security.
- Hides confidential information.
- Different users can have different views.

Difference Between Logical Schema, Physical Schema, and View Schema

Feature	Logical Schema	Physical Schema	View Schema
Meaning	Describes the logical structure of the database.	Describes how data is stored physically.	Describes how users view the database.
Focus	Tables, attributes, and relationships.	Storage files, indexes, and disk organization.	User-specific data presentation.
Storage Details	Not included.	Included.	Not included.
Used By	Database designers and developers.	Database administrators (DBAs).	End users and applications.

Feature	Logical Schema	Physical Schema	View Schema
Purpose	Organize data logically.	Improve storage efficiency and performance.	Provide security and simplify data

---

Conclusion:

A **schema** is the structural design of a database, whereas an **instance** is the actual data stored in it. The **logical schema** defines the database structure, the **physical schema** explains how data is stored, and the **view schema** provides customized views of the database to different users. Understanding these concepts is essential for designing, managing, and using databases efficiently.

2. Bank Management System – ER Diagram

Aim

To identify the **entities, attributes, and relationships** in a **Bank Management System** and construct an **Entity Relationship (ER) Diagram**.

a. Entities and Attributes

1.Customer

- Customer_ID (Primary Key)
- Name
- Address
- Phone_Number
- Email
- Date_of_Birth

2. Account

- Account_Number (Primary Key)
- Account_Type
- Balance

- Opening_Date
- Customer_ID (Foreign Key)

3. Branch

- Branch_ID (Primary Key)
- Branch_Name
- Branch_Address
- IFSC_Code
- Manager_Name

4. Employee

- Employee_ID (Primary Key)
- Employee_Name
- Designation
- Salary
- Phone_Number
- Branch_ID (Foreign Key)

5. Transaction

- Transaction_ID (Primary Key)
- Transaction_Date
- Transaction_Type
- Amount
- Account_Number (Foreign Key)

6. Loan

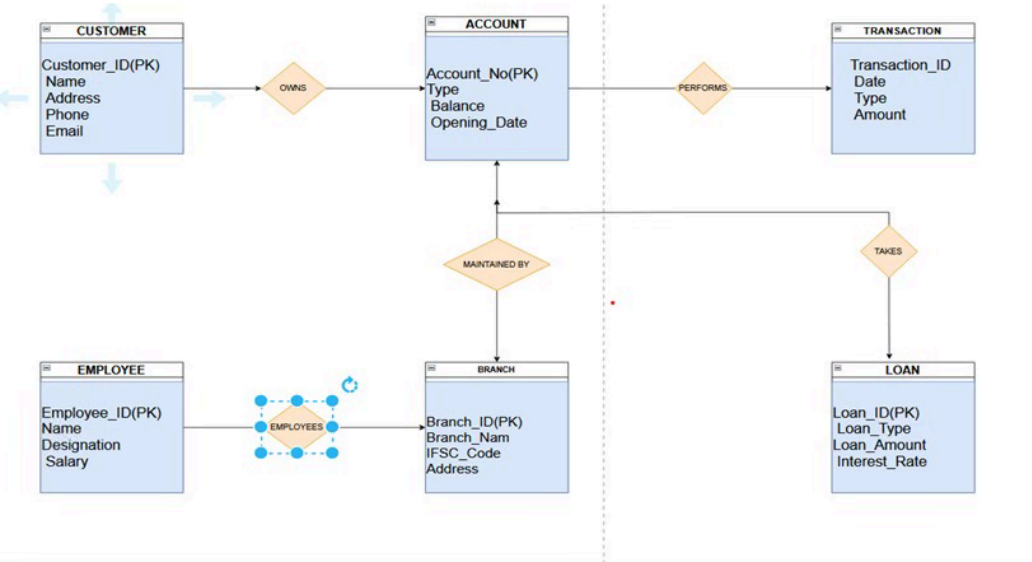
- Loan_ID (Primary Key)
- Loan_Type
- Loan_Amount
- Interest_Rate
- Customer_ID (Foreign Key)

b. Relationships

Relationship	Description	Cardinality

Customer → Account	A customer can have one or more accounts.	1 : M
Branch → Account	A branch maintains many accounts.	1 : M
Branch → Employee	A branch employs many employees.	1 : M
Account → Transaction	One account can have many transactions.	1 : M
Customer → Loan	A customer can take multiple loans.	1 : M

c. ER Diagram :



Result

The **Bank Management System ER Diagram** consists of six main entities: **Customer, Account, Branch, Employee, Transaction, and Loan**. These entities are connected through relationships such as **Customer owns Account, Account performs Transaction, Branch maintains Account, Branch employs Employee, and Customer takes Loan**, providing a clear conceptual model for designing the bank database.

3. Extended ER (EER) Model

The **Extended Entity-Relationship (EER) Model** is an advanced version of the **Entity-Relationship (ER) Model**. It includes additional features to represent complex real-world database systems more accurately.

The EER model supports concepts such as:

- Generalization
- Specialization
- Aggregation
- Inheritance
- Categories (Union Types)

It is widely used in designing large and complex databases.

a. Generalization

Definition

Generalization is the process of combining two or more lower-level entities that have common attributes into a higher-level entity called a **superclass**.

Example

Suppose there are two entities:

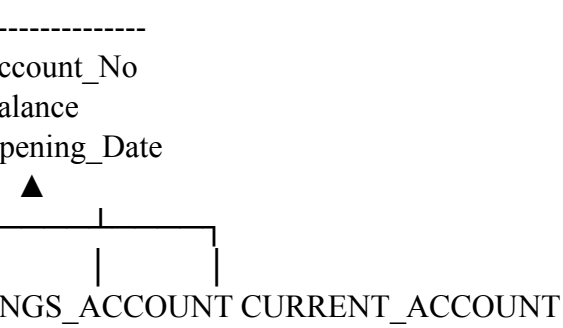
- Savings Account
- Current Account

Both have common attributes like:

- Account Number
- Balance
- Opening Date

These can be generalized into a superclass called **Account**.

Diagram



Advantages

- Reduces data redundancy.
- Improves data organization.

- Makes database design simpler.

b. Specialization

Definition

Specialization is the process of dividing a higher-level entity (superclass) into smaller lower-level entities (subclasses) based on their specific characteristics.

Example

A **Person** entity can be divided into:

- Student
- Employee

Common attributes:

- Person_ID
- Name
- Address

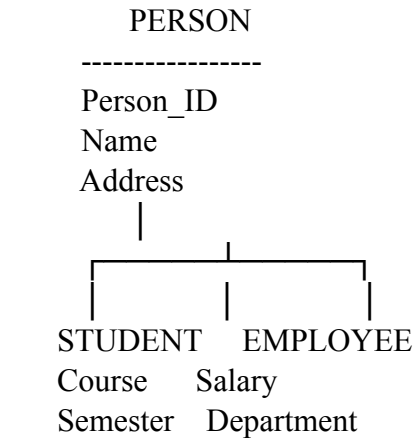
Student-specific:

- Course
- Semester

Employee-specific:

- Salary

Diagram



Advantages

- Represents specific properties clearly.
- Makes data easier to manage.

- Supports inheritance.

c. Aggregation

Definition

Aggregation is the process of treating a relationship between entities as a higher-level entity so that it can participate in another relationship.

It is useful when a relationship itself needs to have another relationship.

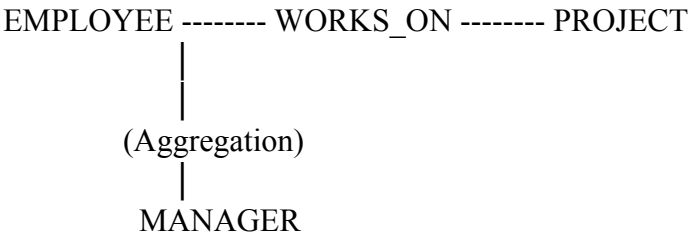
Example

An **Employee** works on a **Project**.

The relationship **Works_On** is treated as one entity.

A **Manager** supervises this work.

Diagram



Advantages

- Represents complex relationships.
- Simplifies database design.
- Improves clarity in large systems.

Advantages of the EER Model

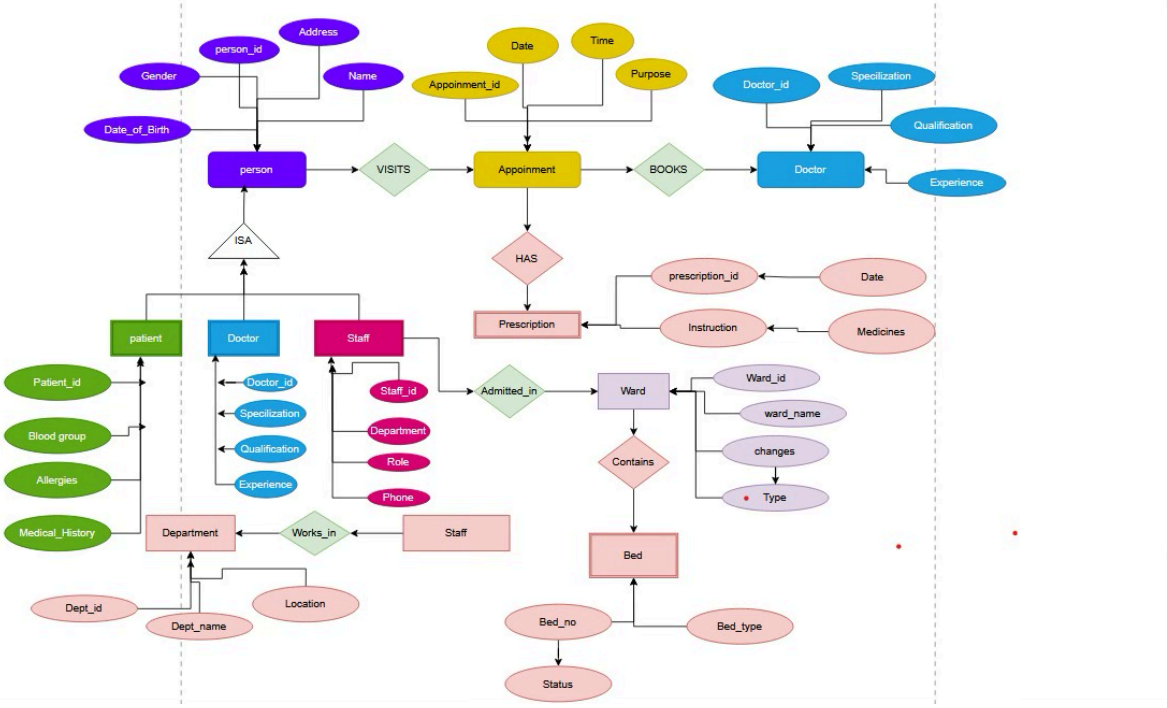
- Represents complex real-world situations.
- Supports inheritance of attributes.
- Reduces redundancy.
- Improves database organization.
- Makes database design more flexible and efficient.

Result

The **Extended ER (EER) Model** extends the basic ER model by introducing advanced concepts such as **Generalization, Specialization, and Aggregation**. These concepts help represent

complex database structures more accurately, reduce redundancy, improve organization, and support efficient database design.

4. EER diagram for a Hospital Management System



5. Role of a DBA (Database Administrator)

A Database Administrator (DBA) is responsible for managing, maintaining, securing, and ensuring the efficient operation of an organization's databases. The DBA ensures that data is available, accurate, secure, and properly backed up.

Responsibilities of a DBA

- **Database Installation and Configuration –**
  - ✓ Installs and configures database management systems such as MySQL, Oracle, or SQL Server.
- **Database Design and Creation –**

- ✓ Helps design database structures, tables, relationships, and storage methods.

- **Data Security –**
  - ✓ Protects the database from unauthorized access by managing users, roles, and permissions.
- **Backup and Recovery –**
  - ✓ Regularly backs up data and restores it in case of system failure or data loss.
- **Performance Monitoring –**
  - ✓ Monitors database performance and improves the speed and efficiency of database operations.
- **Data Integrity –**
  - ✓ Ensures that the stored data is accurate, consistent, and reliable.
- **User Management –**
  - ✓ Creates user accounts and assigns appropriate access privileges.
- **Database Maintenance –**
  - ✓ Performs regular updates, troubleshooting, and maintenance of the database system.
- **Disaster Recovery –**
  - ✓ Develops plans to recover databases after hardware failure, cyberattacks, or other disasters.
- **Technical Support –**
  - ✓ Provides support to developers and users for database-related problems.

Conclusion

The DBA plays an important role in database management by ensuring database security, availability, reliability, and performance.